

Docklite

Aplicación web para la gestión de
contenedores Docker



Autor: Aimar Merodo Alba

Desarrollo de Aplicaciones Web
Curso 2025–2026



Índice de contenidos

1	Introducción	2
2	Objetivos del proyecto	2
2.1	Objetivo general	2
2.2	Objetivos específicos	2
3	Especificación de requisitos	3
3.1	Actores del sistema	3
3.2	Requisitos funcionales.....	3
3.3	Requisitos no funcionales	3
4	Diseño del sistema	4
4.1	Arquitectura general	4
4.2	Modelo de datos	5
4.3	Casos de uso	6
4.4	Diseño de la base de datos.....	7
5	Fases del proyecto.....	8
5.1	Fase 0 - Preparación del entorno	8
5.2	Fase 1 - Autenticación y gestión de usuarios	8
5.3	Fase 2 - Gestión de contenedores.....	9
5.4	Fase 3 - Gestión de imágenes, volúmenes y redes	9
5.5	Fase 4 - Desarrollo del Frontend	9
5.6	Fase 5 - Pruebas y despliegue	9
5.7	Fase 6 - Documentación y memoria.....	9
6	Implementación	10
6.1	Tecnologías y frameworks.....	10
6.1.1	Capa de presentación.....	10
6.1.2	Capa lógica - Backend.....	10
6.1.3	Capa de datos	10
6.1.4	Infraestructura y despliegue	11
6.1.5	Resumen de las capas	11



1 Introducción

Docker se ha convertido en una herramienta ampliamente utilizada para el despliegue de aplicaciones mediante contenedores. Su uso permite aislar aplicaciones y facilitar su distribución en distintos entornos, siendo especialmente común en servidores y plataformas de alojamiento.

Sin embargo, la gestión de Docker se realiza principalmente a través de línea de comandos, lo que puede dificultar su utilización en entornos donde no se dispone de una interfaz gráfica, como servidores remotos o VPS. Además, soluciones como Docker Desktop están orientadas al uso local y no resultan adecuadas para la gestión remota de contenedores en servidores accesibles desde Internet.

Con el objetivo de facilitar el despliegue y la gestión de aplicaciones contenedorizadas en servidores, surge Docklite, una aplicación web que proporciona una interfaz gráfica accesible desde cualquier lugar. Docklite está dirigida a empresas y usuarios individuales que deseen desplegar aplicaciones en su propio servidor y administrarlas de forma sencilla mediante un navegador web, incorporando además un sistema de usuarios que permite gestionar y compartir los recursos de manera controlada.

2 Objetivos del proyecto

2.1 Objetivo general

Desarrollar una aplicación web que permita el despliegue y la gestión de aplicaciones contenedorizadas mediante Docker a través de una interfaz gráfica accesible de forma remota.

2.2 Objetivos específicos

- Permitir el registro y la autenticación de usuarios en la aplicación.
- Facilitar el despliegue de contenedores Docker mediante una interfaz web.
- Gestionar contenedores Docker, permitiendo su creación, visualización y eliminación.
- Gestionar volúmenes Docker asociados a los contenedores.
- Gestionar redes Docker utilizadas por las aplicaciones desplegadas.
- Garantizar el aislamiento y control de acceso a los recursos Docker mediante un sistema de usuarios.
- Permitir el acceso remoto a la gestión de contenedores desde cualquier ubicación a través de un navegador web.



3 Especificación de requisitos

3.1 Actores del sistema

Docklite cuenta con dos actores principales:

- **Administrador:** usuario con permisos para gestionar todos los contenedores y recursos Docker del sistema, independientemente del usuario propietario.
- **Usuario:** persona autenticada que puede gestionar únicamente los recursos Docker que ha creado dentro de la aplicación.

3.2 Requisitos funcionales

- **RF-01:** El sistema permitirá el registro de nuevos usuarios.
- **RF-02:** El sistema permitirá la autenticación de usuarios mediante credenciales.
- **RF-03:** El sistema permitirá diferenciar roles entre usuario y administrador.
- **RF-04:** El sistema permitirá a los usuarios gestionar únicamente los contenedores Docker creados por ellos.
- **RF-05:** El sistema permitirá al administrador gestionar todos los contenedores y recursos Docker del sistema, independientemente del usuario propietario.
- **RF-06:** El sistema permitirá crear, listar y eliminar contenedores Docker.
- **RF-07:** El sistema permitirá crear, listar y eliminar volúmenes Docker.
- **RF-08:** El sistema permitirá crear, listar y eliminar redes Docker.
- **RF-09:** El sistema permitirá la gestión de contenedores a través de una interfaz web accesible de forma remota.

3.3 Requisitos no funcionales

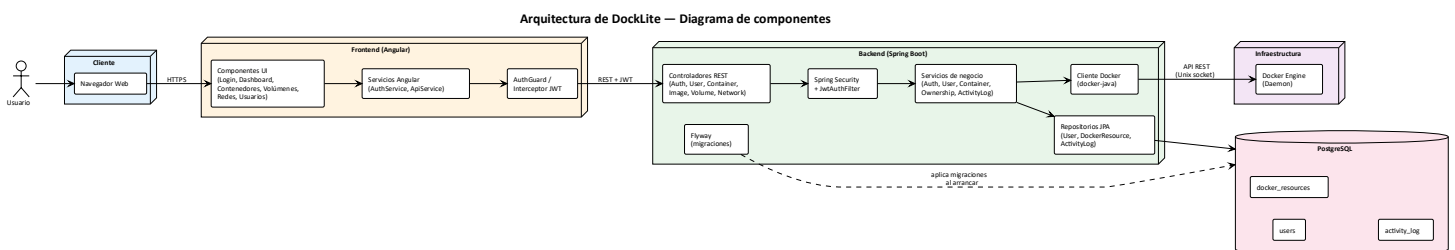
- **RNF-01 (Seguridad):** El acceso al motor Docker se realizará exclusivamente desde el backend de la aplicación.
- **RNF-02 (Seguridad):** El sistema garantizará el aislamiento de los recursos Docker entre los distintos usuarios.
- **RNF-03 (Usabilidad):** La aplicación será accesible desde un navegador web sin necesidad de software adicional.
- **RNF-04 (Portabilidad):** El sistema deberá poder desplegarse mediante contenedores Docker.
- **RNF-05 (Escalabilidad):** La arquitectura de la aplicación permitirá la incorporación de nuevas funcionalidades en el futuro.
- **RNF-06 (Rendimiento):** Las operaciones básicas de gestión de recursos Docker deberán ejecutarse en un tiempo razonable para el usuario.



4 Diseño del sistema

4.1 Arquitectura general

Docklite tiene una arquitectura **cliente – servidor** en tres capas: frontend, backend y base de datos. Esta separación facilita el mantenimiento y permite que el proyecto sea escalable a largo plazo.



El sistema se compone de cinco bloques:

- **Cliente:** navegador web, único punto de entrada
- **Frontend:** Angular SPA (Single Page Application) con componentes de UI, servicios HTTP y un interceptor que añade el JWT a cada petición.
- **Backend:** Java con Spring Boot, arquitectura en capas (controladores -> servicios -> repositorios). Spring Security valida el JWT en cada petición mediante un filtro personalizado. La librería Docker-java es el único componente autorizado a comunicarse con el motor Docker.
- **Base de datos:** PostgreSQL con tres tablas, users, docker_resources y activity_log. Gestionadas mediante migraciones versionadas con Flyway.
- **Docker Engine:** recibe las órdenes del backend a través de su API local. El navegador nunca accede a él directamente

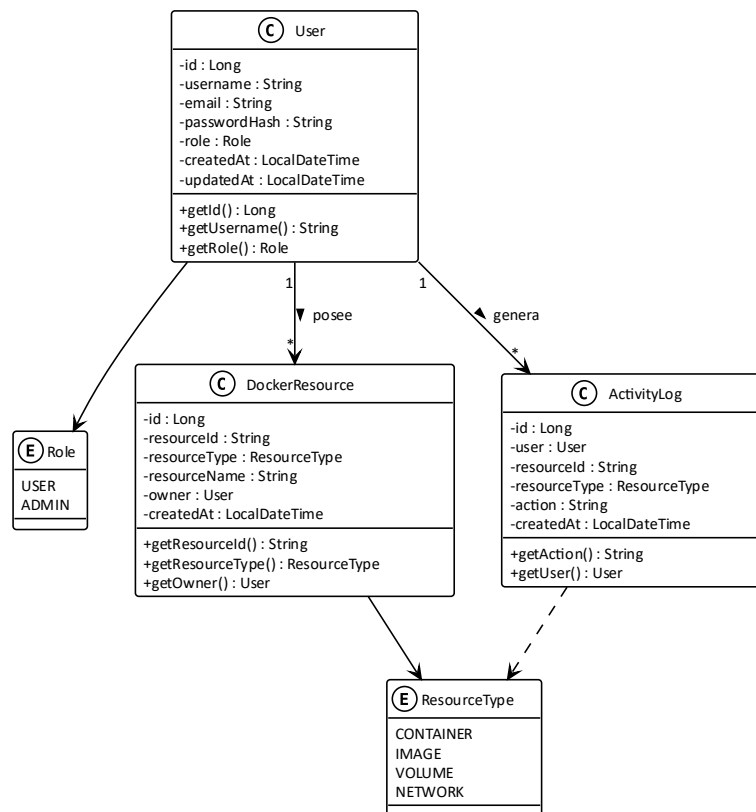
El control de propiedad se aplica en la capa de servicios: cuando un usuario lista recursos, el backend filtra la respuesta del Daemon de Docker para devolver únicamente aquellos registrados a su nombre en la tabla docker_resources, salvo que tenga rol ADMIN, en ese caso ve todos los contenedores del sistema. Este sistema garantiza el aislamiento de cada recurso.



4.2 Modelo de datos

El modelo de negocio de Docklite se articula en torno a tres entidades principales: User, DockerResource y ActivityLog.

Diagrama de clases — Modelo de negocio



User: representa a un usuario de la aplicación. Contiene credenciales (email y hash de contraseña) y un rol (**USER** o **ADMIN**) que determina sus permisos.

DockerResource: registra la propiedad de cada recurso Docker creado a través de la aplicación. Su campo `resourceId` guarda el identificador que Docker asigna al recurso, y `resourceType` indica de qué tipo se trata (contenedor, imagen, volumen o red). Un usuario puede poseer varios recursos, y una misma imagen puede tener varios propietarios, ya que Docker mantiene una única copia en disco, aunque la pulleen distintos usuarios.

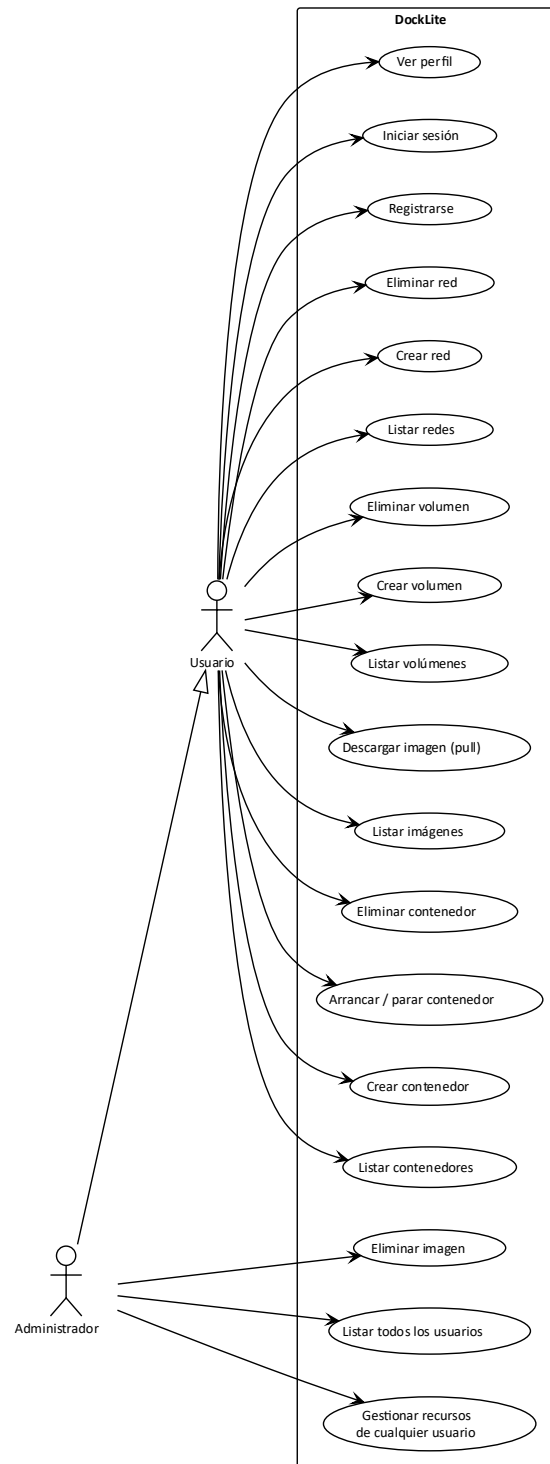
ActivityLog: almacena el historial de acciones realizadas por cada usuario sobre los recursos Docker, permitiendo auditar el uso del sistema.



4.3 Casos de uso

Se han identificado dos actores en el sistema: el **Usuario** estándar, que gestiona únicamente sus propios recursos, y el **Administrador**, que hereda todos los casos del usuario y añade capacidades de gestión global.

Diagrama de casos de uso — DockLite





Los casos de uso se agrupan en cinco bloques funcionales:

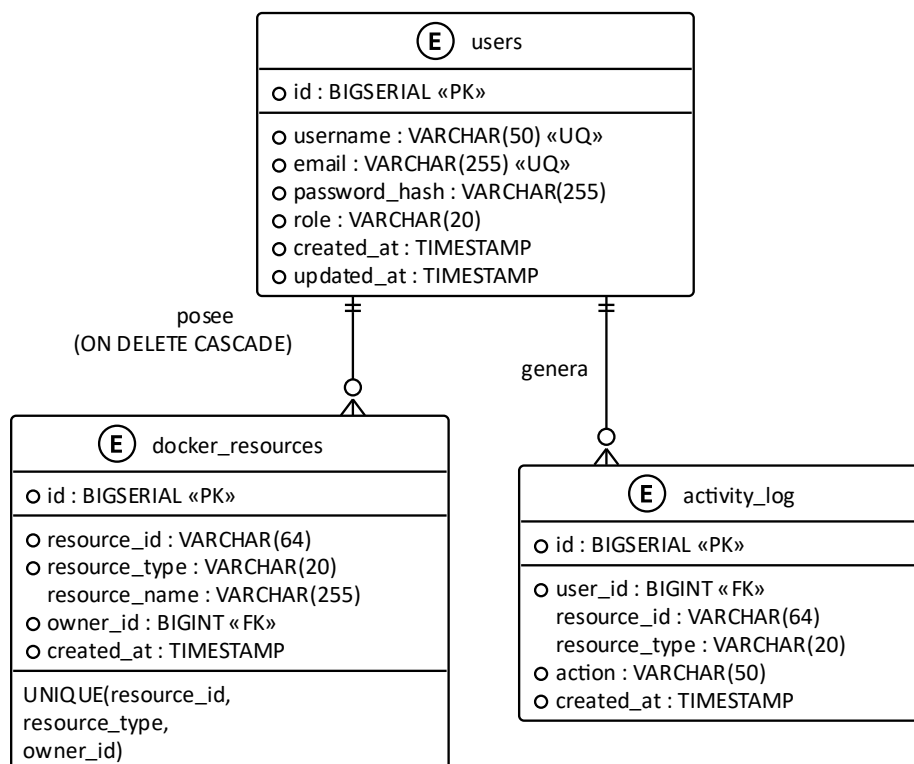
- **Autenticación:** registro, inicio de sesión y consulta del perfil propio.
- **Contenedores:** listar, crear, arrancar, parar y eliminar.
- **Imágenes:** listar y descargar (pull). La eliminación queda reservada al administrador, ya que Docker mantiene una única copia de cada imagen en disco y borrar una podría afectar a otros usuarios.
- **Volúmenes y redes:** listar, crear y eliminar los propios.
- **Administración:** listar todos los usuarios y gestionar los recursos Docker de cualquiera de ellos, saltándose el filtro de propiedad.

La herencia entre actores (Administrador → Usuario) refleja que el rol ADMIN dispone de todas las capacidades del rol USER además de las suyas propias, evitando duplicar casos en el diagrama.

4.4 Diseño de la base de datos

La base de datos de Docklite se ha diseñado con tres tablas que cubren las necesidades del modelo: gestión de usuarios, control de propiedad sobre los recursos Docker y registro de auditoría.

Diagrama Entidad-Relación — DockLite





- **users:** datos de los usuarios registrados. El rol (USER o ADMIN) se guarda como texto por legibilidad.
- **docker_resources:** asocia cada recurso Docker con su propietario. La restricción UNIQUE(resource_id, resource_type, owner_id) permite que varios usuarios "posean" la misma imagen (Docker guarda una única copia en disco) pero impide duplicados por usuario. Usa ON DELETE CASCADE para limpiar recursos al borrar un usuario.
- **activity_log:** registra las acciones mutantes para auditoría. Sin CASCADE, para conservar el histórico, aunque se borre el usuario.

El esquema se gestiona con migraciones versionadas en **Flyway**, aplicadas automáticamente al arrancar el backend.

5 Fases del proyecto

El desarrollo de Docklite se ha organizado siguiendo una **metodología ágil iterativa**, dividiendo el trabajo en fases cortas y entregables que permiten validar el progreso de forma continua. Cada fase se ha definido con un objetivo concreto y un conjunto de funcionalidades asociadas a los requisitos identificados en el apartado anterior.

5.1 Fase 0 - Preparación del entorno

Configuración inicial del repositorio, elección del stack tecnológico y puesta en marcha del entorno de desarrollo. Incluye la instalación de Java 21, Spring Boot, PostgreSQL mediante Docker Compose, la estructura del proyecto Maven y la conexión básica entre el backend y la base de datos. Al finalizar esta fase, el esqueleto del proyecto arranca correctamente y las tres tablas del modelo (users, docker_resources, activity_log) se crean automáticamente mediante migraciones Flyway.

5.2 Fase 1 - Autenticación y gestión de usuarios

Implementación del sistema de registro, inicio de sesión y autenticación basada en **JSON Web Tokens**. Se desarrollan las entidades JPA, los repositorios, los endpoints REST de autenticación y el filtro de seguridad que valida el token en cada petición.



5.3 Fase 2 - Gestión de contenedores

Integración con el motor Docker a través de la librería docker-java. Se implementan los endpoints para listar, crear, arrancar, parar y eliminar contenedores, introduciendo además el mecanismo de **control de propiedad** que filtra los recursos visibles para cada usuario.

5.4 Fase 3 - Gestión de imágenes, volúmenes y redes

Extensión del patrón de ownership al resto de recursos Docker. Se añaden los endpoints para gestionar imágenes (con la particularidad de que la eliminación queda reservada al administrador), volúmenes y redes.

5.5 Fase 4 - Desarrollo del Frontend

Construcción de la interfaz en **Angular**, incluyendo las pantallas de login, dashboard, gestión de contenedores, imágenes, volúmenes, redes y administración de usuarios. Se implementa el interceptor HTTP que añade el token JWT a cada petición y el AuthGuard que protege las rutas privadas.

5.6 Fase 5 - Pruebas y despliegue

Validación manual de los endpoints mediante Postman, pruebas de integración entre Frontend y backend, y configuración del despliegue mediante Docker Compose. Se verifica el cumplimiento de los requisitos no funcionales, especialmente el aislamiento entre usuarios (RNF-02) y el acceso exclusivo al daemon desde el backend.

5.7 Fase 6 - Documentación y memoria

Redacción de la memoria del proyecto, preparación de la presentación y revisión final del código y la documentación.

Fase	Duración estimada
Fase 0 - Preparación del entorno	1 semana
Fase 1 - Autenticación y gestión de usuarios	1 semana
Fase 2 - Gestión de contenedores	1-2 semanas
Fase 3 - Gestión de imágenes, volúmenes y redes	1-2 semanas
Fase 4 - Desarrollo del Frontend	2 semanas
Fase 5 - Pruebas y despliegue	1 semana
Fase 6 - Documentación y memoria	A lo largo del proyecto



6 Implementación

6.1 Tecnologías y frameworks

La elección de las tecnologías de Docklite se ha realizado priorizando la madurez del ecosistema, la compatibilidad con estándares abiertos y la productividad durante el desarrollo.

6.1.1 Capa de presentación

Angular (v21) Framework para Single Page Applications con sistema de componentes, enrutado y cliente HTTP con interceptores, utilizados para añadir el JWT a cada petición. typescript aporta tipado estático y mejor mantenimiento.

6.1.2 Capa lógica - Backend

Spring Boot 4 / Java 21. Framework de referencia en Java. Su autoconfiguración reduce el código repetitivo y acelera el desarrollo. Java 21 aporta mejoras en rendimiento.

Spring Security + jjwt. Gestiona la autorización por roles y el filtro de seguridad. La librería jjwt se encarga de la creación y validación de los tokens JWT.

docker-java. Cliente Java oficial para comunicarse con el daemon Docker mediante su API REST. Evita tener que ejecutar comandos del sistema y facilita las pruebas.

6.1.3 Capa de datos

PostgreSQL 16. SGBD elegido por su madurez, su soporte de constraints complejas (necesarias para el UNIQUE compuesto de docker_resources) y su buena integración con Spring Data JPA.

Spring Data JPA + Hibernate. Implementación del estándar JPA. Spring Data genera automáticamente los repositorios a partir de interfaces, reduciendo el código necesario para acceder a los datos.

Flyway. Gestiona las migraciones de la base de datos mediante ficheros .sql versionados, aplicados automáticamente al arrancar el backend. Garantiza que cualquier entorno tenga el mismo esquema.

Lombok. Genera getters, setters, constructores y builders mediante anotaciones.



6.1.4 Infraestructura y despliegue

Docker Compose + Maven. Docker Compose levanta la base de datos y el frontend con un único comando, garantizando un entorno reproducible y mantenible. Maven gestiona las dependencias y el ciclo de construcción del backend.

6.1.5 Resumen de las capas

Capa	Tecnología	Justificación
Presentación	Angular v21	SPA con tipado estático e interceptor JWT
Backend	Spring Boot 4 / Java 21	Autoconfiguración y ecosistema maduro
Seguridad	Spring Security + jjwt	Autenticación JWT y control por roles
Datos	Spring Data JPA + Hibernate	Repositorios autogenerados
BBDD	PostgreSQL 16	Robustez y soporte de constraints avanzadas
Migraciones	Flyway	Control de versiones del esquema
Docker	docker-java	Cliente oficial para el daemon
Despliegue	Docker Compose + Maven	Entorno reproducible